

# A Microservices-Based Architectural Approach to System Observability and Resilience

## 1. Abstract

This paper presents the design and implementation of a distributed system structured around microservices architecture to address the challenges of scalability and observability in modern software engineering. The project utilizes three core functional services—**User, Order, and Payment**—to demonstrate inter-service communication and fault tolerance. By leveraging **Docker Compose** for orchestration, and integrating the **ELK Stack (Elasticsearch, Logstash, Kibana)** alongside **Zipkin** for distributed tracing, the study provides a comprehensive framework for monitoring system health. A key contribution of this work is the analysis of a controlled failure scenario, which illustrates how distributed systems behave under stress and how logging mechanisms can be utilized for rapid root-cause analysis in a stateless environment.

## 2. Introduction

The transition from monolithic architectures to microservices has become a paradigm shift in software development, driven by the need for continuous deployment and independent scaling. However, the decentralized nature of microservices introduces "operational blindness," where tracking a single transaction across multiple network boundaries becomes a complex task.

This project aims to bridge the gap between theoretical microservice concepts and practical implementation. By simulating a typical e-commerce workflow—comprising user validation, order management, and payment processing—we explore the life cycle of a request in a containerized environment. The primary objective is not only to ensure functional correctness but also to implement a robust monitoring infrastructure that provides real-time insights into system performance and failure modes.

## 3. Background and Related Work

### 3.1. Microservices and Statelessness

Microservices are designed as small, autonomous services that communicate over well-defined APIs. In this project, a **Stateless Architecture** was preferred. By managing data in-memory, the system avoids the overhead of external database synchronization during the prototyping phase, allowing for higher throughput and simplified container orchestration.

### 3.2. Observability: Tracing and Logging

Observability is defined through three pillars: metrics, logs, and traces.

- **Distributed Tracing (Zipkin):** In a distributed system, a single user action may trigger a chain of calls. Zipkin allows us to assign a unique *Trace ID* to each request, visualizing the latency and status of every hop between the User, Order, and Payment services.

- **Log Aggregation (ELK Stack):** Centralized logging is critical. Instead of inspecting individual Docker container logs, Logstash collects data, Elasticsearch indexes it, and Kibana provides a visual interface for complex queries.

### 3.3. Containerization with Docker

Docker provides the necessary isolation to ensure that the development environment is identical to the production environment. Using **Docker Compose**, we define the entire infrastructure as code, ensuring that network configurations and environment variables are standardized across all services.

## 4. Development and Implementation Details

### 4.1. Functional Components

The system architecture is divided into three functional domains:

1. **User Service:** Acting as the identity provider, it handles requests for user profiles and validates credentials before any order is initiated.
2. **Order Service:** The orchestrator of the business logic. It receives requests from the User Service, validates the order details, and initiates a synchronous call to the Payment Service.
3. **Payment Service:** This service processes transaction requests. It is designed to simulate various financial response codes (Success, Declined, or Error).

### 4.2. Intentional Fault Injection and Resilience Testing

To simulate real-world instability, a **Chaos Engineering** element was introduced. A controlled failure mechanism was implemented in the Payment Service's logic. Under specific conditions—such as a specific "Product ID" or randomized intervals—the service returns a **500 Internal Server Error**. This allows the development team to:

- Observe how the **Order Service** handles downstream failures.
- Verify if the **Zipkin** dashboard correctly identifies the Payment Service as the "bottleneck" or "point of failure."
- Analyze the stack trace within **Kibana** to understand the exception's origin without manual intervention.

### 4.3. Infrastructure Orchestration

The entire ecosystem is orchestrated via a `docker-compose.yaml` file. This includes:

- **Network Isolation:** A dedicated virtual bridge network for service-to-service communication.
- **Resource Management:** Ensuring that the ELK stack, which is resource-intensive, has sufficient memory to index logs while the microservices remain responsive.
- **Environment Standardization:** All services utilize Spring Boot's native support for Zipkin and Logback to ensure seamless data flow to the monitoring tools.

## **5. Conclusion**

This project demonstrates that while microservices increase architectural complexity, the use of proper orchestration and observability tools can mitigate these risks. The integration of User, Order, and Payment services within a Docker-managed ELK and Zipkin environment proves that real-time monitoring and fault analysis are essential for maintaining system reliability in distributed environments.